

DataBases

Task-1.Create an AWS Lambda function in Python that adds a new item to a DynamoDB table called 'MusicPlaylists' every time it receives a song name and artist via a test event.

Step 1: Create the DynamoDB table

Open the AWS Management Console:

[AWS Management Console](#)

Search for:

DynamoDB

Open **DynamoDB**.

Make sure you're using the same AWS Region as your Lambda. Since you've been using Mumbai, you can keep:

Asia Pacific (Mumbai) — ap-south-1

Click:

Create table

Enter:

Table name

MusicPlaylists

Partition key

Enter:

songName

Select:

String

So your table configuration should look like:

Table name: MusicPlaylists

Partition key:

songName

String

You don't need a sort key for this assignment.

Leave the remaining settings at their defaults and click:

Create table

Wait until the table status becomes:

Active

Step 2: Create the Lambda function

Go to:

AWS Lambda → Functions → Create function

Select:

Author from scratch

Enter:

Function name

MusicPlaylistManager

Runtime

Choose the newest available **Python 3.x** runtime.

For example:

Python 3.13

Leave the other settings at their defaults.

Click:

Create function

Step 3: Give Lambda permission to access DynamoDB

This is an important step.

Your Lambda needs permission to:

- add items to DynamoDB
- read items from DynamoDB

After creating the Lambda, go to:

Configuration → Permissions

Under **Execution role**, click the role name.

This opens IAM.

Choose:

Add permissions → Attach policies

For a simple classroom/lab exercise, you can attach:

AmazonDynamoDBFullAccess

Then save.

Security note

AmazonDynamoDBFullAccess is broader than necessary. For a real application, you should create a least-privilege policy allowing only `dynamodb:PutItem` and `dynamodb:Scan` on `MusicPlaylists`.

For this exercise, the broader managed policy is easier.

Step 4: Write the Lambda code

Go back to:

Lambda → Functions → MusicPlaylistManager

Scroll down to the **Code** section.

You'll see:

```
lambda_function.py
```

Delete the existing code and paste:

```
import json
```

```
import boto3

import os

dynamodb = boto3.resource("dynamodb")

TABLE_NAME = os.environ.get("TABLE_NAME", "MusicPlaylists")
table = dynamodb.Table(TABLE_NAME)


def lambda_handler(event, context):

    print("Received event:")
    print(json.dumps(event))

    action = event.get("action", "add")

    # -----
    # ADD A SONG
    # -----

    if action == "add":

        song_name = event.get("songName")
        artist = event.get("artist")

        # Check for missing fields
        if not song_name or not artist:
```

```
print("ERROR: songName or artist is missing")
```

```
return {  
    "statusCode": 400,  
    "body": json.dumps({  
        "error": "songName and artist are required"  
    })  
}
```

```
print(f"Adding song: {song_name}")
```

```
print(f"Artist: {artist}")
```

```
table.put_item(  
    Item={  
        "songName": song_name,  
        "artist": artist  
    }  
)
```

```
print("Song successfully added to DynamoDB")
```

```
return {  
    "statusCode": 200,  
    "body": json.dumps({  
        "message": "Song added successfully",  
        "songName": song_name,
```

```

        "artist": artist
    })
}

# -----
# LIST ALL SONGS
# -----

elif action == "list":

    print("Reading all songs from DynamoDB")

    response = table.scan()

    songs = response.get("Items", [])

    print(f"Number of songs found: {len(songs)}")

    return {
        "statusCode": 200,
        "body": json.dumps(songs)
    }

# -----
# UNKNOWN ACTION
# -----

else:

```

```
print(f"ERROR: Unknown action: {action}")
```

```
return {  
    "statusCode": 400,  
    "body": json.dumps({  
        "error": "Unknown action"  
    })  
}
```

Click:

Deploy

Step 5: Understand the important parts

DynamoDB connection

```
dynamodb = boto3.resource("dynamodb")
```

This creates a connection to DynamoDB using the Lambda execution role.

Then:

```
table = dynamodb.Table(TABLE_NAME)
```

selects the MusicPlaylists table.

Adding a song

This code:

```
table.put_item(  
    Item={  
        "songName": song_name,  
        "artist": artist
```

```
}  
)
```

creates an item such as:

songName = Perfect

artist = Ed Sheeran

inside DynamoDB.

Step 6: Create the Task 1 test event

At the top of the Lambda page, click:

Test

Choose:

Create new event

Event name:

AddSongTest

Use this JSON:

```
{  
  "action": "add",  
  "songName": "Perfect",  
  "artist": "Ed Sheeran"  
}
```

Save it.

Click:

Test

Step 7: Verify Task 1

You should receive a response similar to:

```
{  
  "statusCode": 200,  
  "body": "{\\\"message\\\": \\\"Song added successfully\\\", \\\"songName\\\": \\\"Perfect\\\", \\\"artist\\\": \\\"Ed Sheeran\\\"}"  
}
```

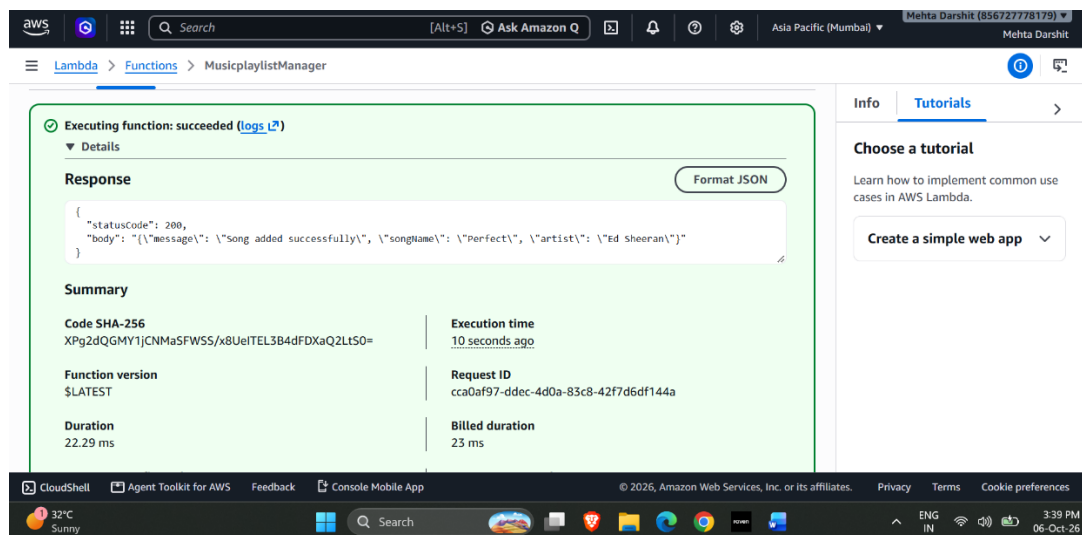
Now go to:

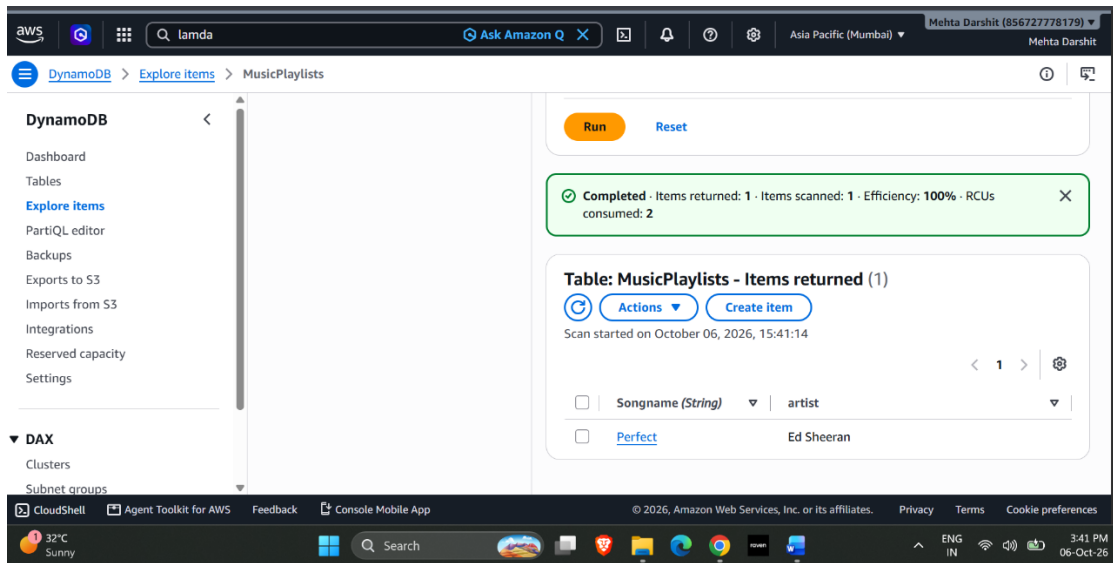
DynamoDB → Tables → MusicPlaylists → Explore table items

You should see something similar to:

songName artist

Perfect Ed Sheeran





Task-2. Modify your Lambda function to read all songs from the 'MusicPlaylists' DynamoDB table and return them as a JSON array.

Go back to:

Lambda → MusicPlaylistManager → Test

Create another test event.

Name it:

ListSongsTest

Use:

```
{  
  "action": "list"  
}
```

Click **Save**.

Then click:

Test

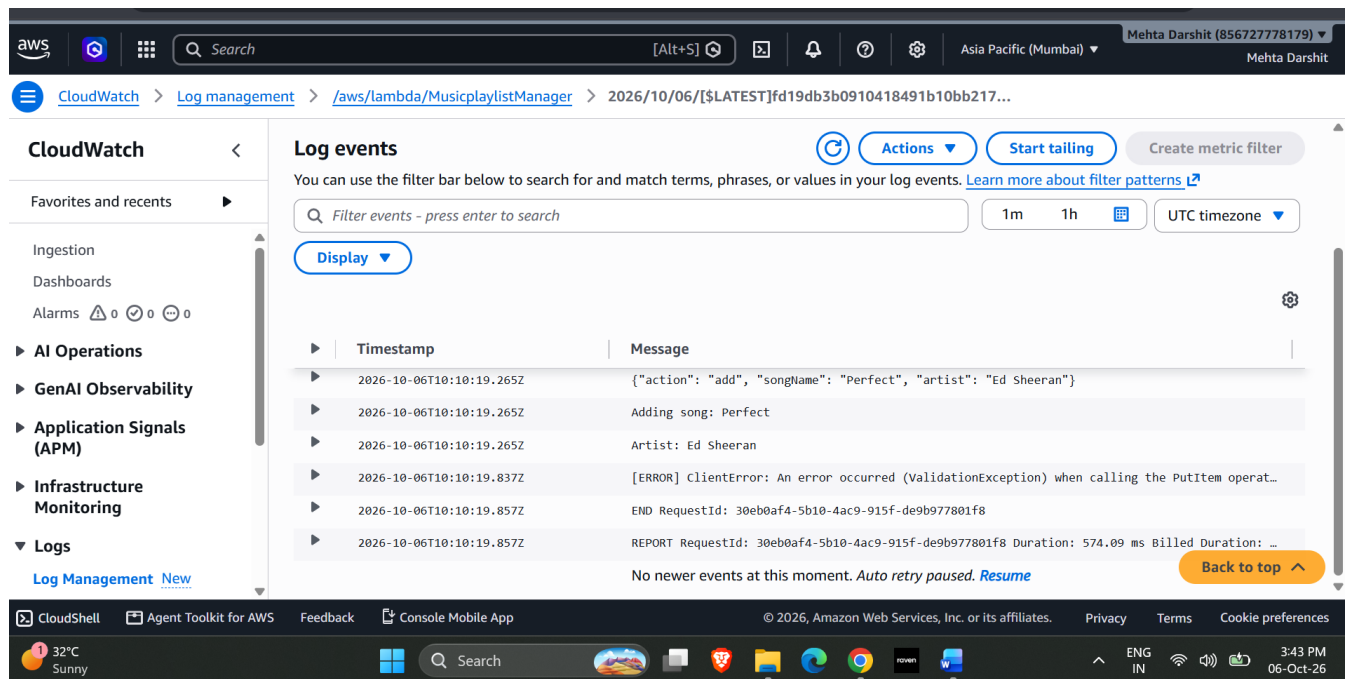
→ Verify the returned JSON

If you previously added:

Perfect - Ed Sheeran

Output:

```
{
  "statusCode": 200,
  "body":
    "[{\"artist\": \"Ed Sheeran\", \"songName\": \"Perfect\"}]"
}
```



Task-3. Set up Amazon CloudWatch Logs for your Lambda function and trigger it with a test event; then, view the logs to confirm that the song name and artist are correctly logged each time the function runs. Use the `print()` statement in Python to log details, and check the CloudWatch Logs console.

Select:

AddSongTest

and click:

Test

You should get a successful execution.

Now click:

Monitor

on the Lambda function page.

Then choose:

View CloudWatch logs

→ Open the Lambda log group

CloudWatch should take you to a log group similar to:

/aws/lambda/MusicPlaylistManager

Click the log group.

Then open the newest **log stream**.

You should see output similar to:

START RequestId: ...

Received event:

```
{"action": "add", "songName": "Perfect", "artist": "Ed Sheeran"}
```

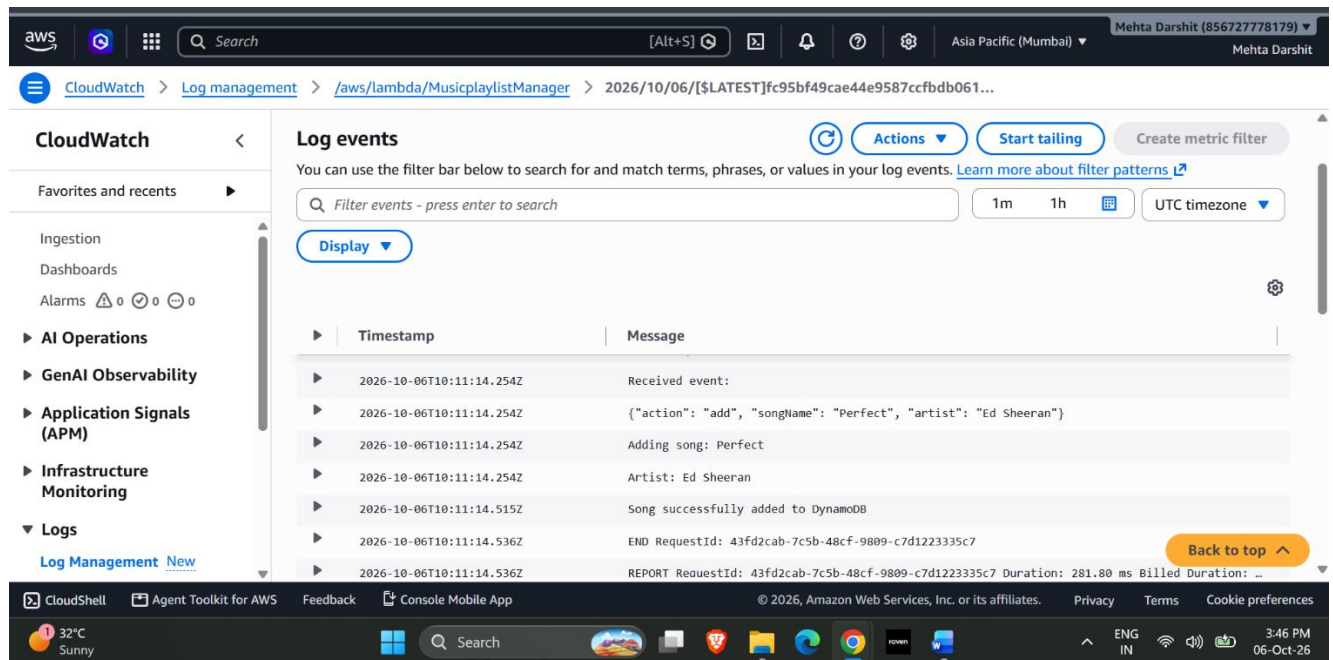
Adding song: Perfect

Artist: Ed Sheeran

Song successfully added to DynamoDB

END RequestId: ...

REPORT RequestId: ...



Task-4. Simulate a bug by intentionally passing an event without the 'artist' field to your Lambda function, then use CloudWatch Logs to debug and fix the error so that the function handles missing fields gracefully.

Create a new test event called:

MissingArtistTest

Use:

```
{  
  "action": "add",  
  "songName": "Shape of You"  
}
```

Notice that:

artist

is missing.

Click:

Save

Then:

Test

Step 13: Check the response

Because we added validation to the Lambda, you should get:

```
{  
  "statusCode": 400,  
  "body": "{\"error\": \"songName and artist are required\"}"  
}
```

This is actually the **fixed behavior**.

The function doesn't crash and doesn't insert an incomplete record.

Step 14: Check CloudWatch

Go to:

Monitor → View CloudWatch logs

Open the newest log stream.

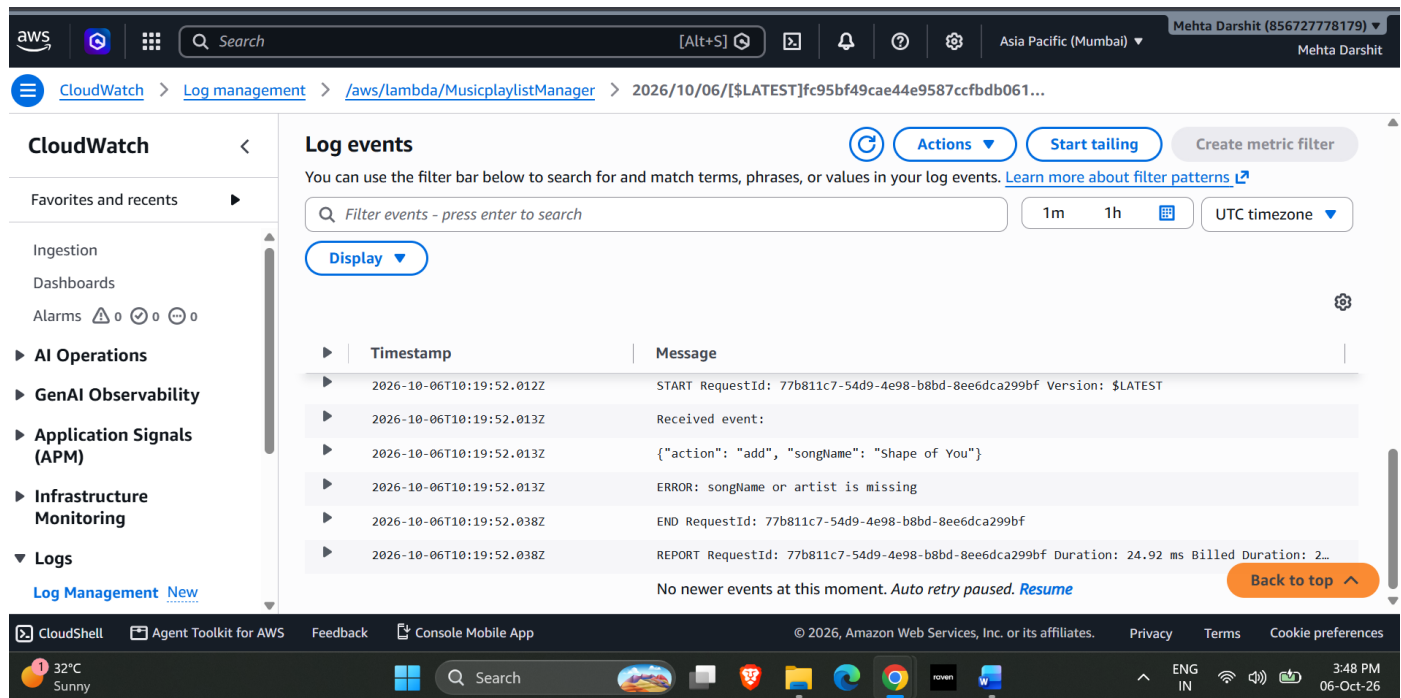
You should see:

Received event:

```
{"action": "add", "songName": "Shape of You"}
```

ERROR: songName or artist is missing

This demonstrates how CloudWatch helps you diagnose the problem.



Task-5. Use ChatGPT to generate a step-by-step checklist for monitoring and debugging a serverless workflow that integrates Lambda with DynamoDB, then apply one suggestion from the checklist to improve your Lambda function's reliability.

Serverless monitoring checklist

1. Check Lambda execution

Confirm:

Lambda → Monitor

Look for:

- Invocation count
- Errors
- Duration
- Throttles

2. Check CloudWatch Logs

Open:

/aws/lambda/MusicPlaylistManager

Check the latest log stream for:

- Input event
- Song name
- Artist
- DynamoDB operation
- Error messages

3. Check DynamoDB

Open:

DynamoDB → MusicPlaylists → Explore table items

Verify that the expected items actually exist.

4. Validate incoming data

Don't assume the event contains every field.

Use:

```
event.get("songName")
```

```
event.get("artist")
```

and validate them before writing to DynamoDB.

5. Handle errors gracefully

Return an appropriate error response rather than allowing the Lambda to crash.

For example:

```
return {  
    "statusCode": 400,  
    "body": json.dumps({  
        "error": "songName and artist are required"  
    })  
}
```

6. Monitor repeated failures

If a Lambda repeatedly fails, use CloudWatch to identify:

- `KeyError`
- `AccessDenied`
- DynamoDB errors
- Invalid input
- Timeout problems